

EasyStay项目 Git操作指南

本指南帮助你完成从项目初始化到GitHub仓库管理的完整流程

目录

1. [项目初始化与GitHub仓库创建](#)
2. [日常开发与提交流程](#)
3. [协作者添加与权限管理](#)
4. [项目维护最佳实践](#)
5. [常见问题解决](#)

1. 项目初始化与GitHub仓库创建

1.1 本地项目初始化Git仓库

打开终端（PowerShell），进入项目根目录：

```
# 进入项目目录
cd D:\360MoveData\Users\胡聪威\Desktop\前端-携程\EasyStay_Project

# 初始化Git仓库
git init
```

成功提示： 你会看到 Initialized empty Git repository in ... 的消息

1.2 配置Git用户信息（首次使用需配置）

```
# 配置用户名（替换为你的GitHub用户名）
git config --global user.name "你的GitHub用户名"

# 配置邮箱（替换为你的GitHub注册邮箱）
git config --global user.email "your.email@example.com"

# 验证配置
git config --list
```

1.3 在GitHub上创建新仓库

步骤：

1. 访问 <https://github.com> 并登录
2. 点击右上角 + 按钮，选择 "New repository"
3. 填写仓库信息：
 - **Repository name:** EasyStay_Hotel_Booking（建议名称）
 - **Description:** 易宿酒店预订平台 - 第五期前端训练营大作业
 - **Visibility:** 选择 **Public**（公开仓库）
 - **Initialize this repository with:** ❌ 不要勾选任何选项（保持空白）
4. 点击 "Create repository"

参考界面位置：

GitHub首页 → 右上角 + → New repository → 填写信息 → Create repository

1.4 将本地项目与远程仓库关联

创建仓库后，GitHub会显示类似下面的命令，复制并在本地执行：

```
# 添加远程仓库（替换为你自己的仓库地址）
git remote add origin https://github.com/你的用户名/EasyStay_Hotel_Booking.git

# 验证远程仓库
git remote -v
```

成功提示： 显示 origin https://github.com/... (fetch) 和 (push)

1.5 首次提交代码到主分支

查看当前文件状态

```
git status
```

添加所有文件到暂存区

```
git add .
```

提交代码（训练营要求：体现开发过程，不要一次性提交）

```
git commit -m "feat: 初始化易宿酒店预订平台项目"
```

- 创建项目基础结构（server/client-mobile/admin-pc）
- 配置后端Express服务器
- 初始化移动端React项目
- 初始化管理端React项目"

推送到远程主分支

```
git push -u origin main
```

注意： 如果GitHub默认分支是 master，使用：

```
git push -u origin master
```

2. 日常开发与提交流程

2.1 检查文件状态

查看哪些文件被修改

```
git status
```

查看详细修改内容

```
git diff
```

查看已暂存的修改

```
git diff --staged
```

2.2 添加文件到暂存区

```
# 添加单个文件
git add 文件名.js

# 添加多个文件
git add 文件1.js 文件2.js

# 添加整个目录
git add src/

# 添加所有修改的文件（常用）
git add .

# 交互式添加（选择性添加）
git add -i
```

2.3 提交变更

```
# 基本提交
git commit -m "提交信息"

# 详细提交（多行）
git commit -m "标题" -m "详细描述"

# 添加并提交（仅限已跟踪文件）
git commit -am "提交信息"
```

2.4 推送到远程主分支

```
# 推送到远程仓库
git push

# 首次推送后，后续只需使用
git push

# 强制推送（谨慎使用！）
git push -f
```

2.5 提交信息规范

格式：

<type>: <subject>

<body>

常用类型：

- feat：新功能
- fix：修复bug
- docs：文档更新
- style：代码格式调整
- refactor：重构
- test：测试相关
- chore：构建/工具相关

示例：

```
git commit -m "feat: 完成移动端酒店列表页面"
```

- 实现酒店卡片展示
- 添加上滑加载更多功能
- 集成筛选和排序功能"

训练营要求： 从项目创建到完成的过程需要在git中有所体现，主分支上每个提交点都应该有明确的意义。

3. 协作者添加与权限管理

3.1 在GitHub界面添加协作者

步骤：

1. 打开你的GitHub仓库页面
2. 点击顶部 **"Settings"** 标签
3. 左侧菜单选择 **"Collaborators"** (协作者)

- 4. 点击 **"Add people"** 按钮
- 5. 输入协作者的GitHub用户名或邮箱
- 6. 选择权限级别，点击 **"Add"**

参考界面位置：

仓库页面 → Settings → Collaborators → Add people

3.2 权限级别说明

权限级别	权限说明	适用场景
Read	只读权限，可克隆、拉取代码	仅查看代码的团队成员
Triage	可管理Issue和PR，但不能推送代码	项目管理人员
Write	可推送代码、创建分支、合并PR	核心开发团队成员
Maintain	可管理仓库设置、部署	技术负责人
Admin	完全控制权限，可删除仓库	仓库所有者

建议设置：

- 训练营队友：**Write** 权限
- 指导教师：**Maintain** 权限

3.3 协作者克隆仓库与提交代码

协作者首次克隆：

```
# 克隆仓库到本地
git clone https://github.com/你的用户名/EasyStay_Hotel_Booking.git

# 进入项目目录
cd EasyStay_Hotel_Booking

# 安装依赖
cd server && npm install
cd ../client-mobile && npm install
cd ../admin-pc && npm install
```

协作者日常提交流程：

1. 开始工作前先拉取最新代码

```
git pull
```

2. 修改代码...

3. 提交修改

```
git add .
```

```
git commit -m "feat: 添加酒店详情页预订功能"
```

4. 推送到远程

```
git push
```

4. 项目维护最佳实践

4.1 定期拉取远程更新

拉取远程更新（推荐每天开始工作前执行）

```
git pull
```

查看远程分支状态

```
git fetch
```

查看本地与远程的差异

```
git log HEAD..origin/main --oneline
```

4.2 解决代码冲突

当 `git pull` 出现冲突时：

1. 查看冲突文件

```
git status
```

2. 打开冲突文件，找到冲突标记：

```
# <<<<<<< HEAD
```

```
# 你的本地代码
```

```
# =====
```

```
# 远程的代码
```

```
# >>>>>>> origin/main
```

3. 手动编辑文件，保留需要的代码，删除冲突标记

4. 标记冲突已解决

```
git add 冲突文件名
```

5. 提交合并

```
git commit -m "merge: 解决代码冲突"
```

6. 推送

```
git push
```

4.3 分支管理建议

简单项目（推荐）：

```
# 所有开发直接在main分支进行
```

```
git add .
```

```
git commit -m "feat: xxx"
```

```
git push
```

复杂项目（多人协作）：


```
# 1. 创建功能分支
git checkout -b feature/酒店搜索功能

# 2. 在分支上开发并提交
git add .
git commit -m "feat: 实现酒店搜索功能"

# 3. 切换回主分支
git checkout main

# 4. 拉取最新代码
git pull

# 5. 合并功能分支
git merge feature/酒店搜索功能

# 6. 推送
git push

# 7. 删除本地分支
git branch -d feature/酒店搜索功能
```

4.4 代码审查流程建议

提交前自查清单：

- ☐ 代码可以正常运行
- ☐ 没有遗留的调试代码（console.log等）
- ☐ 提交信息清晰描述了修改内容
- ☐ 没有提交敏感信息（密码、密钥等）

Pull Request流程（团队项目）：

1. 推送分支到远程
2. 在GitHub创建Pull Request
3. 请求队友审查代码
4. 根据反馈修改
5. 合并到主分支

5. 常见问题解决

5.1 提交错误时的撤销方法

场景1：撤销工作区的修改（未add）

```
# 撤销单个文件的修改
git checkout -- 文件名.js

# 撤销所有修改
git checkout -- .
```

场景2：撤销暂存区的文件（已add未commit）

```
# 将文件从暂存区移回工作区
git reset HEAD 文件名.js

# 撤销所有暂存
git reset HEAD .
```

场景3：撤销最后一次提交（已commit未push）

```
# 保留修改，撤销提交
git reset --soft HEAD~1

# 完全撤销（删除修改）
git reset --hard HEAD~1
```

场景4：修改最后一次提交信息

```
git commit --amend -m "新的提交信息"
```

5.2 本地与远程仓库同步问题

问题：远程有更新，本地也有修改

```
# 方法1: 先提交本地, 再合并远程
git add .
git commit -m "feat: 本地修改"
git pull
git push
```

```
# 方法2: 暂存本地修改, 拉取远程, 恢复本地修改
git stash
git pull
git stash pop
```

问题: 误删了本地文件

```
# 从Git恢复删除的文件
git checkout -- 被删除的文件名
```

5.3 权限错误的排查步骤

错误: 403 Forbidden

```
# 原因1: 未登录GitHub
# 解决: 检查凭据管理器中的GitHub凭据

# 原因2: 协作者权限不足
# 解决: 联系仓库所有者确认权限

# 原因3: 使用HTTPS需要Token
# 解决: 使用Personal Access Token代替密码
```

错误: Repository not found

```
# 原因: 仓库地址错误或没有权限
# 解决:
git remote -v # 检查远程地址
git remote set-url origin https://github.com/正确用户名/正确仓库名.git
```

错误: Permission denied

```
# 原因：SSH密钥问题
# 解决：切换到HTTPS方式
git remote set-url origin https://github.com/用户名/仓库名.git
```

附录：常用Git命令速查表

命令	说明
git init	初始化仓库
git clone <url>	克隆远程仓库
git status	查看状态
git add .	添加所有文件
git commit -m "msg"	提交代码
git push	推送到远程
git pull	拉取远程更新
git log	查看提交历史
git branch	查看分支
git checkout -b <name>	创建并切换分支
git merge <branch>	合并分支
git remote -v	查看远程仓库

训练营提交要求提醒

根据大作业要求：

- 1. ☒ 代码提交到公开的git仓库（GitHub或微信开发者代码管理）
- 2. ☒ 从项目创建到完成的过程需要在git中有所体现
- 3. ☒ 不要做完之后一口气提交上去

4.  主分支上每个提交点都应该有明确的意义

建议提交时间点：

- 项目初始化完成
- 后端API开发完成
- 移动端首页完成
- 移动端列表页完成
- 移动端详情页完成
- 管理端功能完成
- 整体测试完成

文档版本： v1.0

创建日期： 2026-02-15

适用项目： EasyStay易宿酒店预订平台